Daily Currency Exchange ETL

Apache Airflow with Astro CLI, Frankfurter v2, MongoDB and Mongo Express

Prepared for: **saadzafar0**
Report generated: 12 November 2026
Repository: github.com/saadzafar0/airflow-etl

Objective

Implement a daily three-task pipeline that extracts exchange rates, validates and transforms the response, and loads six-field records into a MongoDB database running locally in Docker. Named Airflow XCom keys transfer data between tasks.

Requirement	Implementation
Astro CLI / Airflow	Astro project and pinned Runtime Dockerfile
Exactly three tasks	Extract > Transform > Load
Named XCom transfer	raw_exchange_rates; transformed_exchange_rates
Local database and admin UI	MongoDB + Mongo Express Docker services
Networking / persistence	Internal bridge; named data/config volumes
Repeatable loading	Unique pair/date index and MongoDB upserts
Submission	Source, evidence screenshots, PDF, incremental Git commits

Evidence policy: source images are browser captures of the checked-in files; output images show saved command/API results. Runtime completion is reported only when supported by actual execution evidence.

The three ETL tasks and named XComs

Each task explicitly pushes or pulls its named XCom. Pulls identify the producing task. Automatic return-value XComs are disabled, so the implementation visibly satisfies the assignment's named-transfer requirement.

Source excerpt | dags/currency_exchange_etl.py | lines 17-48

```
17
18
19  def extract_currency_exchange_rates():
20      base, targets = currency_config(
21          os.getenv("BASE_CURRENCY", "EUR"), os.getenv("TARGET_CURRENCIES", "USD,GBP,JPY")
22      )
23      payload = extract_rates(base, targets)
24      get_current_context()["ti"].xcom_push(key=RAW_XCOM_KEY, value=payload)
25      log.info("Extracted %d exchange rates", len(payload["rates"]))
26
27
28  def transform_exchange_rate_data():
29      ti = get_current_context()["ti"]
30      payload = ti.xcom_pull(task_ids="extract_currency_exchange_rates", key=RAW_XCOM_KEY)
31      base, targets = currency_config(
32          os.getenv("BASE_CURRENCY", "EUR"), os.getenv("TARGET_CURRENCIES", "USD,GBP,JPY")
33      )
34      records = transform_rates(payload, base, targets)
35      ti.xcom_push(key=TRANSFORMED_XCOM_KEY, value=records)
36      log.info("Transformed records: %s", records)
37
38
39  def load_data_into_mongodb():
40      ti = get_current_context()["ti"]
41      records = ti.xcom_pull(task_ids="transform_exchange_rate_data", key=TRANSFORMED_XCOM_KEY)
42      summary = load_rates(
43          records, os.environ["MONGO_URI"],
44          os.getenv("MONGO_DATABASE", "currency_exchange"),
45          os.getenv("MONGO_COLLECTION", "exchange_rates"),
46      )
47      ti.xcom_push(key=LOAD_XCOM_KEY, value=summary)
48      log.info("MongoDB load summary: %s", summary)
```

Figure 1. Actual DAG task functions showing extraction, named XCom transfer, transformation and MongoDB loading.

Daily schedule and task dependencies

The DAG runs at 07:15 Asia/Karachi (02:15 UTC). It has exactly three sequential PythonOperator tasks, two retries with a one-minute delay, one active run, and catchup disabled. The rate date always comes from the API response.

Source excerpt | dags/currency_exchange_etl.py | lines 51-79

```python
with DAG(
    dag_id="currency_exchange_etl",
    description="Daily Frankfurter currency rates to local MongoDB",
    start_date=pendulum.datetime(2026, 11, 1, tz="Asia/Karachi"),
    schedule="15 7 * * *",
    catchup=False,
    max_active_runs=1,
    default_args={"owner": "saadzafar0", "retries": 2, "retry_delay": timedelta(minutes=1)},
    tags=["etl", "currency", "mongodb", "frankfurter"],
    doc_md="""
    Runs daily at **07:15 Asia/Karachi (02:15 UTC)**. Uses the latest published
    rate date from Frankfurter, including on weekends and holidays.
    Named XComs: `raw_exchange_rates` -> `transformed_exchange_rates` -> `mongo_load_summary`.
    MongoDB uses a unique pair/date index and upserts for retry-safe loading.
    """,
) as dag:
    extract = PythonOperator(
        task_id="extract_currency_exchange_rates",
        python_callable=extract_currency_exchange_rates, do_xcom_push=False,
    )
    transform = PythonOperator(
        task_id="transform_exchange_rate_data",
        python_callable=transform_exchange_rate_data, do_xcom_push=False,
    )
    load = PythonOperator(
        task_id="load_data_into_mongodb",
        python_callable=load_data_into_mongodb, do_xcom_push=False,
    )
    extract >> transform >> load
```

Figure 2. DAG configuration and Extract > Transform > Load dependencies.

Extract the latest published exchange rates

The extractor requests EUR rates for USD, GBP and JPY from the public Frankfurter v2 endpoint. It checks HTTP status and response shape, applies explicit timeouts, and adds a timezone-aware UTC retrieval timestamp. Currency configuration is validated before sending the request.

Source excerpt | include/currency_etl.py | lines 1-44

```python
"""HTTP extraction, strict transformation, and retry-safe MongoDB loading."""

from datetime import date, datetime, timezone
import math
import re

import requests
from pymongo import ASCENDING, MongoClient, UpdateOne

API_URL = "https://api.frankfurter.dev/v2/rates"
SOURCE = "Frankfurter API"
RAW_XCOM_KEY = "raw_exchange_rates"
TRANSFORMED_XCOM_KEY = "transformed_exchange_rates"
LOAD_XCOM_KEY = "mongo_load_summary"


def currency_config(base="EUR", targets="USD,GBP,JPY"):
    """Normalize configuration and reject invalid or duplicate currency codes."""
    base = base.strip().upper()
    quotes = [code.strip().upper() for code in targets.split(",")]
    if not re.fullmatch(r"[A-Z]{3}", base) or not quotes:
        raise ValueError("Base and targets must be three-letter currency codes")
    if any(not re.fullmatch(r"[A-Z]{3}", code) for code in quotes):
        raise ValueError("Targets must be comma-separated three-letter currency codes")
    if len(quotes) != len(set(quotes)) or base in quotes:
        raise ValueError("Targets must be unique and different from the base")
    return base, quotes


def extract_rates(base, targets):
    """Retrieve the latest published rates; Airflow handles transient retries."""
    response = requests.get(
        API_URL, params={"base": base, "quotes": ",".join(targets)}, timeout=(10, 30)
    )
    response.raise_for_status()
    rows = response.json()
    if not isinstance(rows, list) or not rows:
        raise ValueError("Frankfurter returned an empty or invalid rates array")
    return {
        "rates": rows,
        "retrieved_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
    }


```

Figure 3. API endpoint, currency normalization, HTTP extraction and raw XCom key definitions.

Transform and validate the data contract

The transformation produces the six required fields, keeps the source publication date, and preserves the extraction timestamp. It rejects incomplete currency coverage, duplicate targets, wrong bases, invalid dates and invalid numeric rates. Validation completes before loading begins.

Source excerpt | include/currency_etl.py | lines 45-82

```python
def transform_rates(payload, base, targets):
    """Validate complete API results and map them to the assignment's six fields."""
    if not isinstance(payload, dict) or not isinstance(payload.get("rates"), list):
        raise ValueError("Missing raw_exchange_rates XCom payload")
    retrieved_at = payload.get("retrieved_at")
    if not isinstance(retrieved_at, str):
        raise ValueError("Missing retrieval timestamp")
    timestamp = datetime.fromisoformat(retrieved_at)
    if timestamp.tzinfo is None:
        raise ValueError("Retrieval timestamp must include a timezone")
    records, seen = [], set()
    for row in payload["rates"]:
        if not isinstance(row, dict):
            raise ValueError("Each API rate must be an object")
        quote, rate = row.get("quote"), row.get("rate")
        if row.get("base") != base or quote not in targets or quote in seen:
            raise ValueError("Unexpected base, target, or duplicate currency in API response")
        if isinstance(rate, bool) or not isinstance(rate, (int, float)):
            raise ValueError("Exchange rate must be numeric")
        if not math.isfinite(rate) or rate <= 0:
            raise ValueError("Exchange rate must be finite and positive")
        rate_date = row.get("date")
        if not isinstance(rate_date, str) or date.fromisoformat(rate_date).isoformat() !=
rate_date:
            raise ValueError("Rate date must use YYYY-MM-DD")
        if rate_date > timestamp.date().isoformat():
            raise ValueError("Rate date cannot be later than retrieval date")
        records.append({
            "base_currency": base,
            "target_currency": quote,
            "exchange_rate": float(rate),
            "rate_date": rate_date,
            "source": SOURCE,
            "retrieved_at": retrieved_at,
        })
        seen.add(quote)
    if seen != set(targets):
        raise ValueError("API response is missing one or more requested target currencies")
    return sorted(records, key=lambda record: record["target_currency"])
```

Figure 4. Transformation logic implementing the required MongoDB document schema.

Load safely into local MongoDB

Loading creates a unique compound index on base currency, target currency and publication date. Upserts update an existing record or insert a new date/pair. Repeating the same run therefore preserves one document per pair/date. The task publishes a named summary with actual insert and match counts.

Source excerpt | include/currency_etl.py | lines 85-108

```
 85  def load_rates(records, uri, database="currency_exchange", collection="exchange_rates"):
 86      """Upsert by pair and publication date so retries cannot create duplicates."""
 87      if not isinstance(records, list) or not records:
 88          raise ValueError("Missing transformed_exchange_rates XCom records")
 89      with MongoClient(uri, serverSelectionTimeoutMS=10000) as client:
 90          client.admin.command("ping")
 91          target = client[database][collection]
 92          target.create_index(
 93              [("base_currency", ASCENDING), ("target_currency", ASCENDING), ("rate_date",
ASCENDING)],
 94              unique=True, name="unique_currency_pair_date",
 95          )
 96          result = target.bulk_write([
 97              UpdateOne(
 98                  {key: record[key] for key in ("base_currency", "target_currency", "rate_date")},
 99                  {"$set": record}, upsert=True,
100              )
101              for record in records
102          ], ordered=False)
103          return {
104              "database": database, "collection": collection,
105              "processed": len(records), "inserted": result.upserted_count,
106              "matched": result.matched_count, "modified": result.modified_count,
107              "total_documents": target.count_documents({}),
108          }
```

Figure 5. MongoDB connection, unique index, bulk upserts and load summary.

Docker networking and persistent storage

The scheduler joins the private currency-db network while retaining Astro's default network for the public API. MongoDB exposes no host port. Mongo Express is available only at 127.0.0.1:8081. Named volumes persist the database and configuration directories; labels describe these choices.

Source excerpt | docker-compose.override.yml | lines 1-56

```yaml
services:
  scheduler:
    networks:
      - default
      - currency-db

  mongodb:
    image: mongo:8.0
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USERNAME:?Copy .env.example to .env}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD:?Copy .env.example to .env}
    volumes:
      - mongodb-data:/data/db
      - mongodb-config:/data/configdb
    networks:
      - currency-db
    # No host port: MongoDB is accessible only on the local Docker network.
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "quit(db.adminCommand('ping').ok ? 0 : 1)"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 20s
    labels:
      io.assignment.networking: "private currency-db network; no published database port"
      io.assignment.storage: "mongodb-data:/data/db; mongodb-config:/data/configdb"

  mongo-express:
    image: mongo-express:1.0.2-20
    restart: unless-stopped
    depends_on:
      mongodb:
        condition: service_healthy
    environment:
      ME_CONFIG_MONGODB_URL: ${MONGO_URI:?Copy .env.example to .env}
      ME_CONFIG_MONGODB_ENABLE_ADMIN: "true"
      ME_CONFIG_BASICAUTH_ENABLED: "true"
      ME_CONFIG_BASICAUTH_USERNAME: ${MONGO_EXPRESS_USERNAME:?Copy .env.example to .env}
      ME_CONFIG_BASICAUTH_PASSWORD: ${MONGO_EXPRESS_PASSWORD:?Copy .env.example to .env}
    ports:
      - "127.0.0.1:8081:8081"
    networks:
      - currency-db
    labels:
      io.assignment.networking: "localhost-only web interface on port 8081"
      io.assignment.storage: "stateless administration interface; data stored by MongoDB"

networks:
  currency-db:
    driver: bridge
    internal: true

volumes:
  mongodb-data:
  mongodb-config:
```

Figure 6. Actual Compose override: MongoDB, Mongo Express, networking, volumes and metadata labels.

Actual API response and transformed records

The live request returned 3 rows. Retrieval timestamp: 2026-11-12T10:32:31+00:00. The figures below contain actual saved API and transformation output, rather than the sample values in the assignment.

Captured live Frankfurter API response


```
{
  "rates": [
    {
      "date": "2026-11-12",
      "base": "EUR",
      "quote": "GBP",
      "rate": 0.83594
    },
    {
      "date": "2026-11-12",
      "base": "EUR",
      "quote": "JPY",
      "rate": 177.36
    },
    {
      "date": "2026-11-12",
      "base": "EUR",
      "quote": "USD",
      "rate": 1.1147
    }
  ],
  "retrieved_at": "2026-11-12T10:32:31+00:00"
}
```


Figure 7. Live Frankfurter response and extraction timestamp.

Actual transformed records from the live API


```
[
  {
    "base_currency": "EUR",
    "target_currency": "GBP",
    "exchange_rate": 0.83594,
    "rate_date": "2026-11-12",
    "source": "Frankfurter API",
    "retrieved_at": "2026-11-12T10:32:31+00:00"
  },
  {
    "base_currency": "EUR",
    "target_currency": "JPY",
    "exchange_rate": 177.36,
    "rate_date": "2026-11-12",
    "source": "Frankfurter API",
    "retrieved_at": "2026-11-12T10:32:31+00:00"
  },
  {
    "base_currency": "EUR",
    "target_currency": "USD",
    "exchange_rate": 1.1147,
    "rate_date": "2026-11-12",
    "source": "Frankfurter API",
    "retrieved_at": "2026-11-12T10:32:31+00:00"
  }
]
```


Figure 8. Actual records mapped to the required six-field schema.

Tests and runtime verification

All 31 tests passed locally with Airflow installed and in GitHub Actions. Data tests cover invalid configuration, currencies, rates, dates, timestamps and HTTP failures. The real Airflow parser test verifies imports, exactly three tasks, dependency order, timezone, catchup and concurrency settings.

Actual pytest output

```
.............................                                        [100%]
31 passed in 12.42s
```

Figure 9. Actual local pytest output: 31 passed, including the real DAG import test.

Actual Astro / Airflow execution output

```
Astro CLI: 1.47.0; project initialized with astro dev init.

Actual local Airflow parser output:
{
  "airflow_version": "3.3.2",
  "dag_id": "currency_exchange_etl",
  "schedule": "15 7 * * *",
  "timezone": "Asia/Karachi",
  "task_count": 3,
  "catchup": false,
  "tasks": [
    {
      "task_id": "extract_currency_exchange_rates",
      "downstream": [
        "transform_exchange_rate_data"
      ],
      "retries": 2
    },
    {
      "task_id": "transform_exchange_rate_data",
      "downstream": [
        "load_data_into_mongodb"
      ],
      "retries": 2
    },
    {
      "task_id": "load_data_into_mongodb",
      "downstream": [],
      "retries": 2
    }
  ]
}

Full Docker DAG execution and database load remain unverified.
```

Figure 10. Actual Airflow parser output. This validates DAG structure; full execution and database loading remain unverified.

CI evidence: successful GitHub Actions run.

Database verification status

MongoDB and Mongo Express are fully configured in the submitted Compose override. Runtime verification could not finish because registry connections timed out while downloading images. The initial Astro standalone dependency installation also failed; a separate local Airflow 3.3.2 installation later succeeded and all 31 tests passed. No successful database load, idempotency run, Airflow task success or Mongo Express screen is claimed.

Actual local runtime download failures

```
Astro Docker startup
Error: an error was encountered while building the image, see the build logs for details

Astro standalone startup
Error: error installing dependencies: exit status 2

Runtime image download
failed to copy: httpReadSeeker: failed open: failed to do request: Get "https://127.0.0.1:5072/v2/
runtime/blobs/sha256:309f9ca3b1901e78aebf30b6386cd76bb5e2263fb0c5fd297bd5174ecdc3d801": net/http:
timeout awaiting response headers

MongoDB image download
failed to copy: read tcp [local IPv6 address redacted]:45382->[registry IPv6 address redacted]:443:
read: connection timed out
```

Figure 11. Captured errors from the actual runtime and image-download attempts.

Remaining local verification

After the images download, run astro dev start --no-proxy --wait 5m. Unpause and trigger currency_exchange_etl, inspect the three successful task states and named XCom values, then open the currency_exchange / exchange_rates collection in Mongo Express. Trigger a second run and confirm one document per currency pair and publication date.

Docker and incremental delivery

Actual local Docker services, network and volumes

```
$ docker ps --format table {{.Names}}   {{.Status}}     {{.Ports}}
NAMES     STATUS    PORTS
Exit code: 0

$ docker network ls --filter label=com.docker.compose.project=airflow-etl_070678
NETWORK ID   NAME      DRIVER    SCOPE
Exit code: 0

$ docker volume ls --filter label=com.docker.compose.project=airflow-etl_070678
DRIVER    VOLUME NAME
Exit code: 0
```

Figure 13. Actual local Docker service, network and persistent-volume inspection.

Actual incremental Git commit history

```
7cec33c chore: initialize assignment repository
9d9b9ab feat: add daily Astro currency ETL and local MongoDB services
9f73e11 test: cover exchange-rate validation and DAG structure
68e2301 docs: add setup guide and reproducible PDF report tooling
7594afb chore: initialize Astro CLI project metadata and local settings
37ec538 ci: validate ETL contract and real Airflow DAG on pushes
0fc4a99 docs: document verified results and runtime download limitations
655a9c9 fix: use the Airflow 3.3 DAG parser interface in validation
625f9bc docs: record passing local and CI Airflow validation
```

Figure 14. Incremental commits made during implementation.

Reproduce: copy .env.example to .env, run astro dev start, unpause currency_exchange_etl, and trigger it. Open Airflow at localhost:8080 and Mongo Express at 127.0.0.1:8081. Stop with astro dev stop to preserve database volumes.

Results and reference material

Implemented an Astro CLI project with exactly three sequential Airflow tasks and explicitly named XCom keys.

Configured MongoDB and Mongo Express as local Docker services, with an internal network, localhost-only administration, persistent volumes and metadata labels.

Validated live Frankfurter extraction and transformation for EUR to GBP, JPY and USD; saved the actual records and retrieval timestamp.

Ran all 31 tests successfully on the host with Airflow 3.3.2, including the real DAG import/structure test. GitHub Actions also passed all 31 tests (run 43206246635).

Initialized the Git repository on main and pushed incremental commits to the requested GitHub repository.

Docker Compose accepted the override configuration (--no-consistency validates the override without Astro-generated base services).

Limitation: Full Docker-based Astro/Airflow execution and MongoDB loading remain unverified because container image downloads repeatedly timed out. Local Airflow installation and real DAG parsing succeeded.

Limitation: No successful Airflow graph, persisted database documents, retry/idempotency result or Mongo Express interface screenshot is claimed. The report includes genuine source, API, test and environment-output screenshots.

Files to review

dags/currency_exchange_etl.py: orchestration and named XComs.
include/currency_etl.py: extract, validate, transform, load.
docker-compose.override.yml: local database, admin UI, networks, volumes.
tests/: data contract and real DAG import checks.
reports/evidence/: screenshots and captured output.
README.md: setup and verification instructions.

References

Assignment specification

Frankfurter API documentation

Apache Airflow XCom documentation

Astro CLI installation

Astro Runtime release notes

MongoDB official Docker image